Modèles Algorithmiques Fondamentaux : Analyse, Parcours et Recherche sur Structures de Données

Thème principal : Initiation au développement (C++) - Conception algorithmique et modèles de parcours

Synthèse exécutive : Ce cours pose le socle méthodologique indispensable de la conception algorithmique en BUT 1 Informatique. Il formalise l'analyse préalable des mécanismes d'accès aux données (direct, séquentiel, dynamique) et détaille trois grands modèles algorithmiques canoniques pour les structures à accès direct : le parcours complet systématique, le parcours complet conditionné et la recherche de première occurrence. Une attention particulière est portée sur la gestion rigoureuse des bornes d'indices (de 0 à N-1), le principe du pessimisme algorithmique et l'obligation absolue de la double condition d'arrêt.

Typologie et Analyse Préalable des Mécanismes d'Accès aux Données

Avant d'écrire le moindre algorithme, l'informaticien doit analyser le contrat d'interface de la structure de données. Une structure à accès direct (comme un tableau) permet de cibler instantanément n'importe quel élément via son indice. À l'inverse, un flux séquentiel (fichier texte ou CSV) impose de lire les éléments un par un depuis le début, sans possibilité de saut direct. Tenter d'utiliser une notation par indice sur un fichier séquentiel est une erreur conceptuelle critique.

- Structure de données : collection d'éléments de même type (tableaux, chaînes)
- Accès direct : indexation immédiate en temps constant O(1)
- Accès séquentiel : progression pas à pas (début, suivant, fin) sans saut possible
- Incompatibilité structurelle : interdiction formelle d'utiliser des notations indicées sur un flux séquentiel

Cas concret enseignant : *Le professeur met en garde contre l'erreur classique en partiel : écrire T[i] pour lire la i-ème ligne d'un fichier CSV. Le système d'exploitation ne gérant qu'un curseur séquentiel, ce code est sémantiquement faux et entraîne la note éliminatoire sur la question.*

Premier Modèle : Le Parcours Complet avec Traitement Systématique

Ce modèle s'applique lorsque chaque élément d'une collection doit subir une opération inconditionnelle et uniforme (ex : initialisation, affichage global, conversion). Pour un tableau de N éléments indicé à partir de 0, la variable d'indice varie strictement de 0 à N - 1. Dépasser cette borne provoque un arrêt brutal du programme par dépassement mémoire.

- Objectif : appliquer un traitement uniforme à 100% des éléments
- Structure itérative : boucle bornée 'Pour' car le nombre d'itérations est calculable
- Gestion stricte des bornes : de 0 à N - 1 pour un tableau de N éléments
- Erreur critique : dépassement de capacité 'Out of Bounds' si l'on atteint l'indice N

Cas concret enseignant : *Illustration en amphi : 'Afficher l'intégralité d'un tableau de N cases'. L'erreur fréquente est d'écrire 'Pour i allant de 0 à N', provoquant un crash immédiat lors de la lecture de la case T[N] qui n'existe pas.*

Second Modèle : Le Parcours Complet avec Traitement Conditionnel

Ce modèle conserve l'exploration exhaustive du modèle 1 mais filtre les actions à l'aide d'une condition booléenne. Il sert par exemple à compter des occurrences ou extraire des sous-ensembles. Si la condition compare l'élément courant au précédent (T[i] > T[i-1]), la borne initiale doit être ajustée à 1 pour éviter un accès à l'indice -1.

- Parcours exhaustif : inspection de tous les éléments de la collection
- Traitement sélectif : action déclenchée uniquement si un prédicat logique est validé

- Architecture : imbrication d'une structure 'Si' dans une boucle 'Pour'
- Ajustement des bornes : décalage nécessaire si la condition implique des éléments voisins (i-1 ou i+1)

Cas concret enseignant : *Métaphore de la chaîne de montage : chaque pièce défile obligatoirement devant l'inspecteur (parcours exhaustif), mais le tampon de rebut n'est appliqué que si la pièce présente un défaut précis (traitement conditionné).*

Troisième Modèle : La Recherche de Première Occurrence & Ses Variantes

La recherche s'interrompt dès que la cible est localisée. Le principe du pessimisme impose de supposer l'absence de l'élément au départ. L'algorithme nécessite impérativement deux conditions d'arrêt pour éviter les boucles infinies ou les dépassements de bornes. La variante 1 utilise une sortie anticipée pour conserver l'indice exact, tandis que la variante 2 s'appuie sur une condition globale dans le 'Tant que'.

- Pessimisme algorithmique : initialisation de 'trouve' à 'Faux'
- Boucle non bornée : utilisation obligatoire d'un 'Tant que'
- Double condition d'arrêt : succès (élément trouvé) OU échec (fin de collection)
- Variantes : rupture immédiate (indice conservé) vs condition globale en tête de boucle

Cas concret enseignant : *Règle d'or du professeur : 'En informatique, soyez pessimistes ! Partez du principe que la cible n'existe pas.' Un algorithme de recherche présenté avec une seule condition d'arrêt en examen reçoit la note zéro.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Est-ce qu'on peut appliquer le parcours complet avec traitement systématique sur une structure à accès dynamique ?
Réponse du professeur : Oui, à condition d'utiliser les primitives de parcours adaptées fournies par la structure (itérateurs ou fonctions de taille dynamique) tout en respectant strictement les bornes effectives de la collection à l'instant T.

Question étudiant : Pourquoi s'embêter à nous enseigner deux variantes différentes pour la recherche de première occurrence ?
Réponse du professeur : Parce que la variante 1 est indispensable si vous avez besoin de récupérer la position exacte (l'indice) de l'élément trouvé pour le manipuler ensuite, alors que la variante 2 est parfois plus élégante si vous voulez juste vérifier une simple présence booléenne.

Question étudiant : Comment fait-on si la condition dans un parcours complet conditionné dépend de l'élément précédent (par exemple T[i] > T[i-1]) ?
Réponse du professeur : Il faut impérativement décaler la borne de départ de la boucle 'Pour' à i = 1 au lieu de 0. Si vous commencez à 0, l'évaluation de T[-1] provoquera un bug critique ou une violation mémoire.

Question étudiant : Si l'on veut récupérer TOUS les indices correspondant à une valeur donnée (par exemple trouver tous les '5' d'un tableau), quel modèle doit-on utiliser ?
Réponse du professeur : Il faut utiliser le modèle 2 (Parcours complet avec traitement conditionné) et non le modèle 3, car la recherche de première occurrence s'arrête dès le premier succès, alors que vous voulez inspecter toute la collection pour relever toutes les occurrences.

■ Exercice & Démonstration d'Application

Énoncé : Écrire en pseudo-code algorithmique rigoureux l'algorithme permettant d'afficher tous les nombres strictement positifs d'un tableau d'entiers 'T' contenant 'N' éléments.

Résolution pas à pas :
Étape 1 : Analyse du besoin -> Inspection exhaustive d'un tableau à accès direct avec affichage conditionnel -> Modèle 2 (Parcours complet conditionné). Étape 2 : Bornes d'indices -> De 0 à N - 1 pour éviter tout dépassement. Étape 3 : Structure itérative -> Boucle bornée 'Pour i allant de 0 à N - 1' car la taille N est connue. Étape 4 : Filtre et

action -> 'Si T[i] > 0 Alors Afficher(T[i])'.

Résultat : Algorithme AffichagePositifs Données T : Tableau d'entiers N : Entier (taille du tableau) Variables i : Entier Début Pour i allant de 0 à N - 1 Faire Si T[i] > 0 Alors Afficher(T[i]) Fin Si Fin Pour Fin

■ Auto-Évaluation Flash d'Examen

Q : Quel est le risque critique si l'on écrit une boucle 'Pour i allant de 0 à N' sur un tableau de N éléments indicé à partir de 0 ?

R : Un dépassement de capacité mémoire ('Out of Bounds') lors de l'accès à la case inexistante T[N].

Q : Combien de conditions d'arrêt minimales doit comporter la structure itérative d'une recherche de première occurrence ?

R : Exactement deux : le succès de la recherche (élément trouvé) ET l'épuisement de la structure (fin de collection).

Q : Pourquoi qualifie-t-on la démarche d'initialisation d'un algorithme de recherche de première occurrence de 'pessimiste' ?

R : Parce que l'on pose comme postulat initial que la cible est absente de la collection (trouve <- Faux) jusqu'à preuve du contraire.